

ABSTRACT

With the rapid advancement of cloud computing and distributed systems, modern software applications must support high scalability, continuous availability, and secure access for a large number of users. Traditional single-tier or monolithic deployment models often suffer from performance bottlenecks, limited scalability, difficult maintenance, and increased risk of system failures. To overcome these challenges, multi-tier application architecture has become a widely adopted solution, where the presentation layer, business logic layer, and database layer are deployed independently to ensure better modularity, flexibility, and efficient resource utilization.

However, deploying and managing multi-tier applications manually is complex and time-consuming. Lack of proper monitoring may lead to delayed fault detection, reduced system performance, and unexpected downtime. Furthermore, inadequate security mechanisms expose applications to threats such as unauthorized access, data breaches, and network attacks. These issues highlight the need for an automated framework that integrates deployment, monitoring, and security management into a unified system.

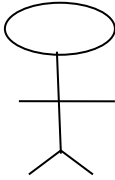
This project proposes a comprehensive approach for deploying and managing multi-tier applications with automated monitoring and security policies. The system utilizes containerization and orchestration technologies to automate deployment and scaling of services across multiple servers. Monitoring tools continuously collect system metrics such as CPU utilization, memory usage, response time, and error rates, enabling real-time performance analysis and proactive fault detection. In addition, security policies including authentication, authorization, encryption, firewall configuration, and role-based access control are enforced to protect sensitive data and ensure safe communication between tiers.

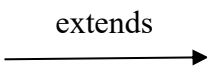
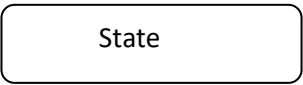
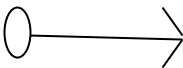
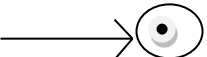
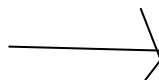
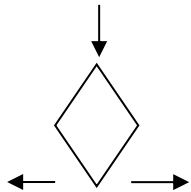
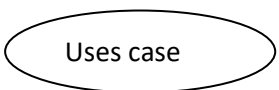
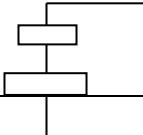
The proposed solution enables efficient resource management, faster deployment, improved reliability, and enhanced security. Experimental evaluation demonstrates reduced downtime, improved response time, and better overall system performance compared to traditional deployment methods. Thus, the framework provides a reliable and scalable platform suitable for enterprise and cloud-based applications.

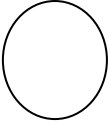
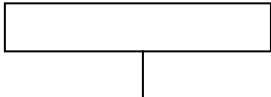
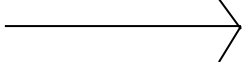
LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO.
2.1	Module Diagram	15
4.1	Use case Diagram	25
4.2	Class Diagram	26
4.3	Object Diagram	27
4.4	State Diagram	28
4.5	Sequence Diagram	29
4.6	Collaboration Diagram	30
4.7	Activity Diagram	31
4.8	Component Diagram	32
4.9	Data flow Diagram	33
4.10	Deployment Diagram	34
4.11	System Architecture	35
7.1	Create Resource Group	45
7.2	Create Virtual Network	45
7.3	Create AppVM Virtual Machine	46
7.4	Create WebVM Virtual Machine	46
7.5	Create DBVM Virtual Machine	47
7.6	Deploy Virtual Machines	47
7.7	Configure Load Balancer	48
7.8	Performance Monitoring	48

LIST OF NOTATIONS

S.NO	NAME	NOTATION	DESCRIPTION
1.	Class	<i>+ public</i> <i>-private</i> <i>Class Name</i> <i>-attribute-</i> <i>attribute</i>	Represents a collection of similar entities grouped together.
2.	Association	NAME Class A Class B Class A Class B	Associations represents static relationships between classes. Roles represents the way the two classes see each other.
3.	Actor		Represents an external entity such as User or Administrator that interacts with the system.
4.	Aggregation	Class A Class B Class A Class B	Represents a "has-a" relationship where one class contains or groups other classes.
5.	Relation (uses)	Uses	Used for additional process communication.

6.	Relation (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.
8.	State		Represents the condition of an object during execution
9.	Initial State		Represents the condition of an object during execution (e.g., Running, Deploying, Failed).
10.	Final state		Represents the termination point of a process or workflow.
11.	Control flow		Shows the sequence of execution or control between activities
12.	Decision box		Represents decision-making or branching based on conditions
13.	Use case		Represents a functional interaction between the system and users.
14.	Component		Represents a modular part of the system such

			as frontend, backend, monitoring service, or security module.
15.	Node		Represents a physical or virtual deployment environment like server or container host.
16.	Data Process/State		In DFD, represents a process that transforms input data into output.
17.	External entity		Represents external systems or users that provide or receive data.
18.	Transition		Represents movement or flow from one state to another.
19.	Object Lifeline		Represents the time duration during which an object participates in a sequence interaction
20.	Message		Represents information exchanged between objects or components.

LIST OF ABBREVIATION

S.NO	ABBREVIATION	EXPANSION
1.	VM	Virtual Machine
2.	VNet	Virtual Network
3.	RG	Resource Group
4.	WebVM	Web Virtual Machine
5.	AppVM	Application Virtual Machine
6.	DBVM	Database Virtual Machine
7.	CPU	Central Processing Unit
8.	LB	Load Balancer
9.	RBAC	Role-Based Access Control
10	OS	Operating System
11.	HTTP	Hyper Text Transfer Protocol
12.	HTTPS	Hyper Text Transfer Protocol Secure
13.	VMSS	Virtual Machine Scale Set

CHAPTER 1

INTRODUCTION

1.1 GENERAL

With the rapid growth of cloud computing and internet-based services, modern applications are expected to handle a large number of users while maintaining high availability, performance, and security. Traditional monolithic systems, where all components of an application run on a single server, often face limitations such as performance bottlenecks, poor scalability, and difficulty in maintenance. To overcome these limitations, modern systems adopt a multi-tier architecture, where different components of the application are separated into multiple layers.

A multi-tier application typically consists of three main layers: the presentation layer (web tier), the application layer (business logic tier), and the data layer (database tier). Each layer performs a specific function and operates independently while communicating with other layers. This separation improves system scalability, maintainability, and performance. For example, the web layer handles user requests, the application layer processes business logic, and the database layer manages data storage.

Cloud platforms such as Microsoft Azure provide powerful tools for deploying and managing multi-tier applications. Azure enables developers to create virtual machines, configure networks, balance traffic across servers, and monitor system performance through a centralized interface. These features make cloud platforms ideal for implementing scalable and secure application architectures.

In this project, a multi-tier application environment is deployed using Microsoft Azure cloud services. The system architecture includes separate virtual machines for the web server, application server, and database server.

These virtual machines are connected through an Azure Virtual Network (VNet) and organized into different subnets to improve security and communication efficiency. A Load Balancer is configured to distribute incoming traffic across servers, ensuring better performance and reliability.

To maintain system health and detect potential issues, Azure Monitor is used to track system performance metrics such as CPU utilization and resource usage. Alerts are generated when predefined thresholds are exceeded, allowing administrators to take corrective actions quickly.

By combining cloud infrastructure, automated monitoring, and security policies, the proposed system demonstrates how multi-tier applications can be deployed and managed efficiently in a cloud environment.

1.2 SCOPE OF THE PROJECT

The scope of this project is to design and implement a cloud-based multi-tier application architecture using Microsoft Azure services. The project focuses on demonstrating how application components can be deployed across multiple servers while ensuring efficient monitoring and security management.

The system architecture includes three main tiers: the web tier, the application tier, and the database tier. Each tier is deployed on a separate virtual machine to maintain modularity and improve scalability. These virtual machines are connected through an Azure Virtual Network and organized into different subnets to maintain secure communication between the layers.

The project also incorporates automated monitoring mechanisms using Azure Monitor. Performance metrics such as CPU utilization and network activity are continuously tracked to ensure that the system operates efficiently. Alerts are generated when abnormal conditions occur, allowing administrators to respond promptly.

Additionally, the project demonstrates the implementation of basic security measures, including controlled access to resources and network isolation between system components. These measures help protect the application infrastructure from unauthorized access.

Overall, the project aims to demonstrate a practical approach to deploying and managing scalable applications using cloud infrastructure

1.3 OBJECTIVE

The main objective of this project is to deploy and manage a multi-tier application architecture using Microsoft Azure cloud services while integrating automated monitoring and security policies.

The specific objectives of the project include:

- To design a multi-tier architecture consisting of web, application, and database layers
- To deploy application components on Azure Virtual Machines
- To configure Azure Virtual Network and subnets for secure communication between system tiers
- To implement a Load Balancer for distributing incoming traffic efficiently
- To monitor system performance using Azure Monitor
- To generate alerts based on system performance thresholds
- To demonstrate secure and scalable cloud-based application deployment

By achieving these objectives, the project highlights how cloud infrastructure can be used to simplify the deployment and management of distributed applications.

1.4 PROBLEM STATEMENT

Modern web applications must support a growing number of users while maintaining high availability, performance, and security. Managing such applications using traditional single-server systems is challenging because these systems often suffer from limited scalability, poor fault tolerance, and increased maintenance complexity.

When all application components are deployed on a single server, heavy workloads can cause system slowdowns or failures. Additionally, monitoring system performance and identifying faults becomes difficult without proper monitoring tools. Security also becomes a concern when applications do not implement proper access control and network isolation mechanisms.

To address these challenges, organizations adopt multi-tier architectures deployed on cloud platforms. However, configuring and managing such architectures manually can be complex without the proper tools and frameworks.

This project addresses these challenges by demonstrating the deployment of a multi-tier application using Microsoft Azure. The system uses virtual machines, network segmentation, load balancing, and automated monitoring to improve system performance and reliability.

1.5 EXISTING SYSTEM

Traditional application deployment methods often rely on single-server architectures or manually managed infrastructure. In such systems, the web interface, application logic, and database components are often hosted on the same server.

While this approach may work for small applications, it becomes inefficient as the number of users increases. High workloads can lead to server overload, slow response times, and service interruptions.

Another limitation of traditional systems is the lack of automated monitoring. Without continuous monitoring, administrators may not detect system failures or performance issues until users experience service disruptions.

Furthermore, managing infrastructure manually increases the risk of configuration errors and security vulnerabilities.

1.1.1 EXISTING SYSTEM DISADVANTAGES

- Limited scalability when user traffic increases
- Higher risk of server overload and system failures
- Lack of automated monitoring and alert mechanisms
- Difficult infrastructure management and configuration
- Increased security risks due to improper network isolation

1.1.2 LITERATURE SURVEY

Cloud computing has become an important platform for deploying and managing modern applications. Organizations use cloud services to improve system scalability, reliability, and performance. Many studies highlight the advantages of using cloud infrastructure for multi-tier application deployment and management.

Research on multi-tier architecture shows that dividing applications into layers such as web, application, and database tiers improves system performance and resource utilization. This layered structure allows the system to handle large workloads efficiently.

Studies on cloud infrastructure also emphasize the role of virtualization technologies. Virtual machines enable applications to run in isolated environments, making resource management easier. Cloud platforms like Microsoft Azure provide flexible services that support scalable application deployment.

Load balancing techniques are another important aspect of cloud systems. By distributing incoming requests across multiple servers, load balancing helps prevent server overload and improves system availability.

Researchers also highlight the importance of monitoring tools in cloud environments. Monitoring services track system performance metrics and generate alerts when resource usage exceeds predefined limits. Security mechanisms such as authentication and access control further protect cloud infrastructure from unauthorized access.

Overall, previous studies show that combining multi-tier architecture, load balancing, monitoring, and security mechanisms improves the efficiency and reliability of cloud-based application

Title: A Survey on Dynamic Load Balancing Techniques in Cloud Computing

Author: Daisy Sharmah, Kanak Bora

Year: 2023.

Description:

This paper presents a comprehensive study of dynamic load balancing techniques in cloud computing environments. Load balancing is an essential mechanism used to distribute workloads across multiple virtual machines to ensure efficient resource utilization. The authors explain that static load balancing methods are less effective in modern cloud systems because they do not adapt to changing workloads. In contrast, dynamic load balancing techniques monitor system conditions in real time and allocate resources accordingly. The paper compares various dynamic load balancing algorithms based on parameters such as response time, throughput, scalability, and fault tolerance. It also highlights challenges such as resource underutilization and uneven workload distribution. The study concludes that dynamic load balancing plays a critical role in improving the performance and reliability of cloud-based applications by ensuring optimal use of available resources

Title: Multi-Cloud Load Distribution for Three-Tier Applications

Author: A. A. Adewojo et al.

Year: 2022.

Description:

This research focuses on deploying three-tier applications in multi-cloud environments. The authors emphasize that dividing an application into three layers—web, application, and database—improves scalability and simplifies system management. Each layer performs a specific function and can be managed independently, which enhances the flexibility of the system. The study also highlights the importance of load distribution across multiple cloud platforms. By distributing workloads across different servers and cloud providers, the system can avoid performance bottlenecks and reduce the risk of failure. The paper explains that proper

load balancing ensures that incoming user requests are efficiently handled, resulting in improved application performance and reliability.

Title: Load Balancing Techniques in Cloud Computing Environment

Author: D. A. Shafiq et al.

Year: 2022.

Description:

This paper provides an analytical review of various load balancing techniques used in cloud computing systems. The authors discuss different algorithms designed to manage incoming user requests and distribute workloads efficiently across multiple servers. The study compares these techniques based on key performance metrics such as response time, throughput, and system efficiency. The paper also identifies several challenges associated with load balancing, including workload imbalance, resource wastage, and increased latency. It emphasizes that selecting an appropriate load balancing algorithm is essential for achieving optimal system performance. The authors suggest that efficient load balancing improves resource utilization, enhances system stability, and reduces operational costs in cloud environments.

Title: Systematic Review of Load Balancing and Task Scheduling in Cloud Computing

Author: N. Devi et al.

Year: 2024.

Description:

This paper presents a systematic review of load balancing and task scheduling techniques in cloud computing. The study examines multiple research works to understand how workload distribution and task scheduling can be optimized to improve system performance. The authors explain that load balancing ensures equal distribution of tasks across available resources, while scheduling determines the order in which tasks are executed. The paper highlights that combining load balancing with effective scheduling strategies can significantly enhance system efficiency. It also discusses the role of emerging technologies such as artificial intelligence and machine learning in improving resource management. These advanced techniques can predict workload patterns and optimize resource allocation dynamically. The study concludes that

integrating intelligent scheduling with load balancing is essential for managing large-scale cloud applications effectively.

1.2 PROPOSED SYSTEM

To overcome the limitations of traditional deployment methods, this project proposes a cloud-based multi-tier architecture implemented using Microsoft Azure services.

In the proposed system, the application is divided into three independent tiers: the web tier, the application tier, and the database tier. Each tier is deployed on separate Azure Virtual Machines, allowing them to operate independently while communicating through a secure virtual network.

The system also uses an **Azure Load Balancer** to distribute incoming traffic across web servers, ensuring better performance and preventing server overload. Automated monitoring is implemented using **Azure Monitor**, which tracks system performance and generates alerts when predefined thresholds are exceeded.

Security policies are applied through controlled network access and user authentication mechanisms, ensuring that only authorized users can access the system resources.

This architecture improves system reliability, scalability, and performance while reducing the complexity of infrastructure management.

1.6.1 PROPOSED SYSTEM ADVANTAGES

- Improved scalability through distributed architecture
- Better performance through load balancing
- Automated monitoring of system performance
- Faster detection of system issues through alert mechanisms
- Improved security through network isolation and access control
- Efficient cloud-based infrastructure management

CHAPTER 2

PROJECT DESCRIPTION

2.1 GENERAL

Cloud computing enables organizations to deploy and manage applications without maintaining physical infrastructure. Modern applications require scalability, reliability, and efficient management of system resources. To achieve these requirements, applications are commonly designed using a multi-tier architecture.

A multi-tier system divides an application into different layers such as the web tier, application tier, and database tier. Each layer performs a specific function and communicates with the others through a secure network. This separation improves system performance, scalability, and maintainability.

In this project, a multi-tier application architecture is implemented using Microsoft Azure cloud services. The system uses Azure Virtual Machines to deploy the web server, application server, and database server. These components are connected through an Azure Virtual Network with separate subnets to maintain secure communication.

To improve system availability and performance, an Azure Load Balancer is used to distribute incoming traffic. In addition, Azure Monitor is configured to track system performance and generate alerts when resource usage exceeds predefined limits.

This approach demonstrates how cloud platforms can efficiently support modern application deployment and management.

2.1 METHODOLOGY

The project implementation follows a structured process using Microsoft Azure services.

First, a Resource Group is created to organize all cloud resources. Then a Virtual Network (VNet) is configured to allow communication between system components. The network is divided into three subnets: Web Subnet, Application Subnet, and Database Subnet.

Virtual machines are deployed within these subnets to represent the three tiers of the application. The WebVM handles user requests, the AppVM processes application logic, and the DBVM stores application data.

An Azure Load Balancer is configured to distribute incoming traffic across the web servers. Finally, Azure Monitor is used to track system performance metrics and generate alerts when CPU usage exceeds the defined threshold.

This methodology ensures efficient deployment, monitoring, and management of the multi-tier application.

2.1.1 MODULES NAME:

This project has the following 6 modules:

Module	Description
1. User Authentication Module	Verifies user credentials before allowing access to the system
2. Multi-Tier Deployment Module	Deploys application components across web, application, and database servers.
3. Network Configuration Module	Configures virtual networks and subnets for secure communication
4. Load Balancing Module	Distributes incoming traffic across servers
5. Monitoring and Alert Module	Tracks system performance and generates alerts
6. Security Management Module	Enforces access control and network security policies

2.1.2 MODULES EXPLANATION AND DIAGRAM

1. User Authentication Module

This module allows administrators or users to securely access the system. Before performing any operations, users must log in using valid credentials. The authentication process ensures that only authorized users can access the cloud resources and manage the application environment. Once authenticated, users can deploy application components, monitor system performance, and manage infrastructure resources.

2. Multi-Tier Deployment Module

This module is responsible for deploying the application architecture in the cloud environment. The system follows a three-tier architecture consisting of the web tier, application tier, and database tier. Each tier is deployed on a separate virtual machine to improve scalability and maintainability. The web server handles user requests, the application server processes business logic, and the database server stores application data.

3. Network Configuration Module

This module establishes the communication infrastructure between system components. An Azure Virtual Network (VNet) is created to connect the servers. The network is divided into different subnets such as WebSubnet, AppSubnet, and DBSubnet. Each subnet hosts a specific server type, ensuring secure communication and network isolation between the tiers.

4. Load Balancing Module

The load balancing module distributes incoming user traffic across available web servers. An Azure Load Balancer ensures that requests are evenly distributed so that no single server becomes overloaded. This improves system performance and enhances application availability during high traffic conditions.

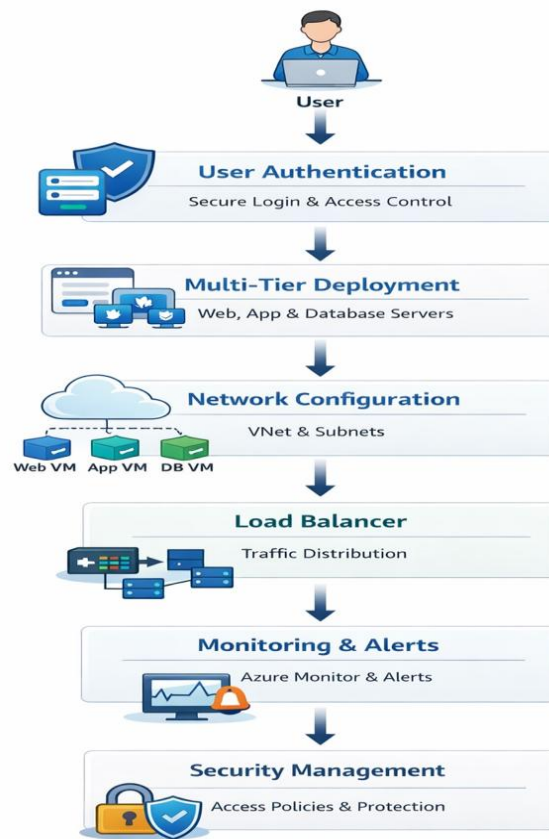


Fig 2.1: Module Diagram

5. Monitoring and Alert Module

This module continuously monitors system performance using Azure Monitor. Metrics such as CPU usage, network activity, and resource utilization are tracked. When predefined thresholds are exceeded, alerts are generated automatically. These alerts notify administrators about potential performance issues so that corrective actions can be taken.

6. Security Management Module

The security module ensures that cloud resources are protected from unauthorized access. It manages authentication, role-based access control, and network security rules. By implementing security policies, the system maintains confidentiality, integrity, and availability of application resources.

2.1.3 GIVEN INPUT EXPECTED OUTPUT:

1. User Authentication Module

Input

- Username and password entered by the user
- Login request from the user interface

Output

- Successful login access to the system dashboard
- Error message if authentication fails

2. Multi-Tier Deployment Module

Input

- Configuration details for virtual machines
- Deployment parameters such as server name, operating system, and region

Output

- Creation of WebVM, AppVM, and DBVM
- Deployment of application components across different tiers

3. Network Configuration Module

Input

- Virtual network configuration details
- Subnet configuration (WebSubnet, AppSubnet, DBSubnet)

Output

- Azure Virtual Network created
- Secure communication established between application tiers

4. Load Balancing Module

Input

- Incoming user requests to the application
- Backend server configuration

Output

- Balanced distribution of user traffic across servers

- Improved system performance and availability

5. Monitoring and Alert Module

Input

- System performance metrics such as CPU usage, memory usage, and network traffic

Output

- Performance monitoring reports
- Alerts generated when resource usage exceeds predefined thresholds

6. Security Management Module

Input

- User access requests
- Security policies and access control rules

Output

- Controlled access to system resources
- Protection of cloud infrastructure from unauthorized users

2.3 TECHNIQUE USED

The system uses a **multi-tier cloud deployment technique** implemented through Microsoft Azure services.

This technique separates application components into independent layers and deploys them on different virtual machines within a cloud infrastructure. Each tier performs a specific role in the system architecture.

The main techniques used in the project include:

Virtualization

Azure Virtual Machines provide virtual computing environments that allow applications to run on cloud-based servers.

Network Segmentation

Azure Virtual Network and subnets are used to organize application tiers and maintain secure communication between them.

Traffic Distribution

Azure Load Balancer distributes incoming user requests across servers to maintain system performance.

Automated Monitoring

Azure Monitor collects performance metrics and generates alerts when system performance deviates from normal conditions.

By combining these techniques, the system achieves reliable application deployment, efficient monitoring, and improved resource management.

CHAPTER 3

REQUIREMENTS ENGINEERING

3.1 GENERAL

This chapter describes the system requirements and specifications needed to implement the project. The system is developed using Microsoft Azure cloud services to demonstrate how application components can be deployed across multiple layers and managed efficiently.

The proposed system follows a three-tier architecture, where the application is divided into the web tier, application tier, and database tier. Each tier is deployed on separate virtual machines within an Azure Virtual Network. This architecture helps improve scalability, performance, and maintainability of the system.

The system also integrates monitoring and security mechanisms. Azure Monitor is used to track system performance and generate alerts when resource usage exceeds predefined limits. Security policies and authentication mechanisms ensure that only authorized users can manage the application infrastructure.

3.2 HARDWARE REQUIREMENTS

Since the project is implemented using cloud infrastructure, the hardware requirements for the user system are minimal.

- Processor : Intel Core i3 or higher
- RAM : Minimum 4 GB
- Hard Disk : Minimum 20 GB free space
- Internet Connection : Stable broadband connection

The cloud infrastructure resources such as servers and networking components are provided through the Microsoft Azure platform.

3.3 SOFTWARE REQUIREMENTS

The following software tools and platforms are used for the implementation of the project.

- Operating System : Windows 10 / Linux
- Cloud Platform : Microsoft Azure
- Web Browser : Google Chrome / Microsoft Edge

- Virtual Machine Operating System : Ubuntu Server
- Monitoring Tool : Azure Monitor

These tools provide the environment required to deploy, manage, and monitor the multi-tier application system.

3.4 FUNCTIONAL REQUIREMENTS

Functional specifications describe the main operations performed by the system.

- The system allows users or administrators to log in securely to access cloud resources.
- A resource group is created to organize and manage all Azure resources used in the project.
- A virtual network is configured to enable communication between system components.
- Subnets are created to separate the web, application, and database layers.
- Virtual machines are deployed in each subnet to host application components.
- A load balancer distributes incoming user traffic across web servers to improve performance.
- Azure Monitor continuously tracks system metrics such as CPU usage and network activity.
- Alerts are generated automatically when system performance exceeds predefined thresholds.

These functional operations ensure efficient deployment and management of the multi-tier application.

3.5 NON-FUNCTIONAL REQUIREMENTS

Non-functional specifications describe the quality attributes and performance characteristics of the system.

Scalability

The cloud-based architecture allows the system to scale resources based on application workload and user demand.

Reliability

The use of load balancing and monitoring services ensures that the system remains available and performs consistently.

Security

Authentication mechanisms and access control policies protect system resources from unauthorized access.

Performance

The multi-tier architecture distributes application tasks across different servers, improving system efficiency and response time.

Maintainability

Cloud infrastructure simplifies system management by allowing administrators to monitor resources and configure services through a centralized platform.

CHAPTER 4

DESIGN ENGINEERING

4.1 GENERAL

System design is an important phase in software development that focuses on defining the overall architecture and structure of the proposed system. It describes how different components of the system interact with each other and how the system processes data to achieve the required functionality.

In this project, the system design is based on a multi-tier cloud architecture deployed using Microsoft Azure. The architecture is divided into three main layers: the web tier, application tier, and database tier. Each tier is deployed on separate virtual machines to improve scalability, security, and performance. The system also includes additional components such as the load balancer for traffic distribution and Azure Monitor for performance monitoring and alert generation.

The design phase includes several UML diagrams such as Use Case Diagram, Class Diagram, Object Diagram, State Chart Diagram, Sequence Diagram, Activity Diagram, Collaboration Diagram, Component Diagram, Deployment Diagram, and Data Flow Diagram. These diagrams provide a clear understanding of the system structure, interactions between components, and the workflow of the application.

Overall, the system design helps in visualizing the complete architecture of the proposed system and ensures that the application can be deployed and managed efficiently in the Azure cloud environment.

4.1.1 USECASE DIAGRAM

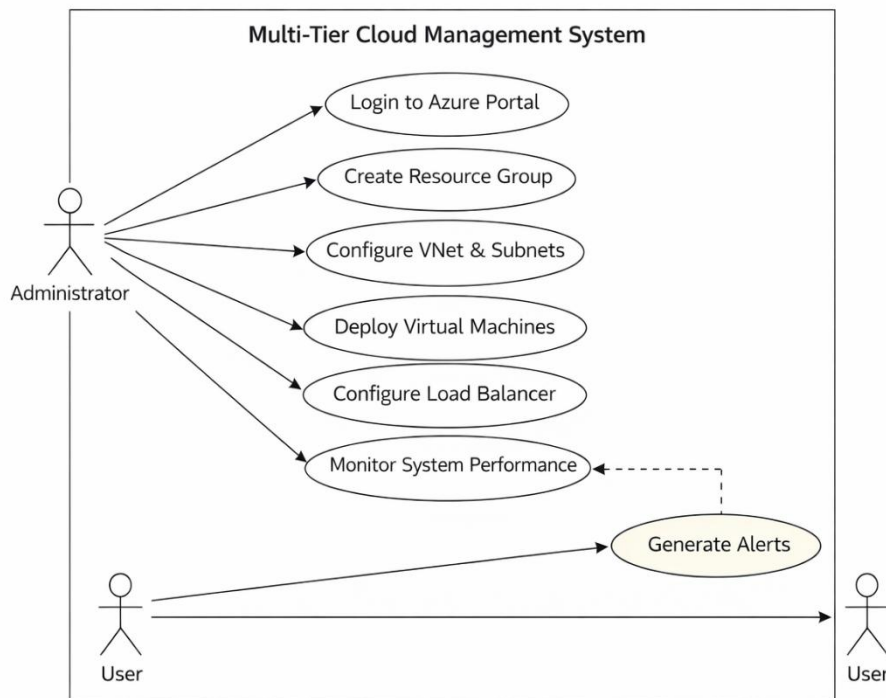


Fig 4.1: Use case diagram

EXPLANATION:

The Use Case Diagram illustrates the interaction between the **Administrator** and the **User** in the multi-tier cloud management system. The administrator manages the cloud infrastructure by logging into the Azure portal, creating resource groups, configuring virtual networks and subnets, deploying virtual machines, and setting up the load balancer. The system performance is continuously monitored using **Azure Monitor**, which generates alerts when abnormal activity is detected. Users interact with the system by accessing the application services deployed on the cloud infrastructure. This diagram shows how different system functionalities are coordinated to manage and monitor the multi-tier application environment.

4.1.2 CLASS DIAGRAM

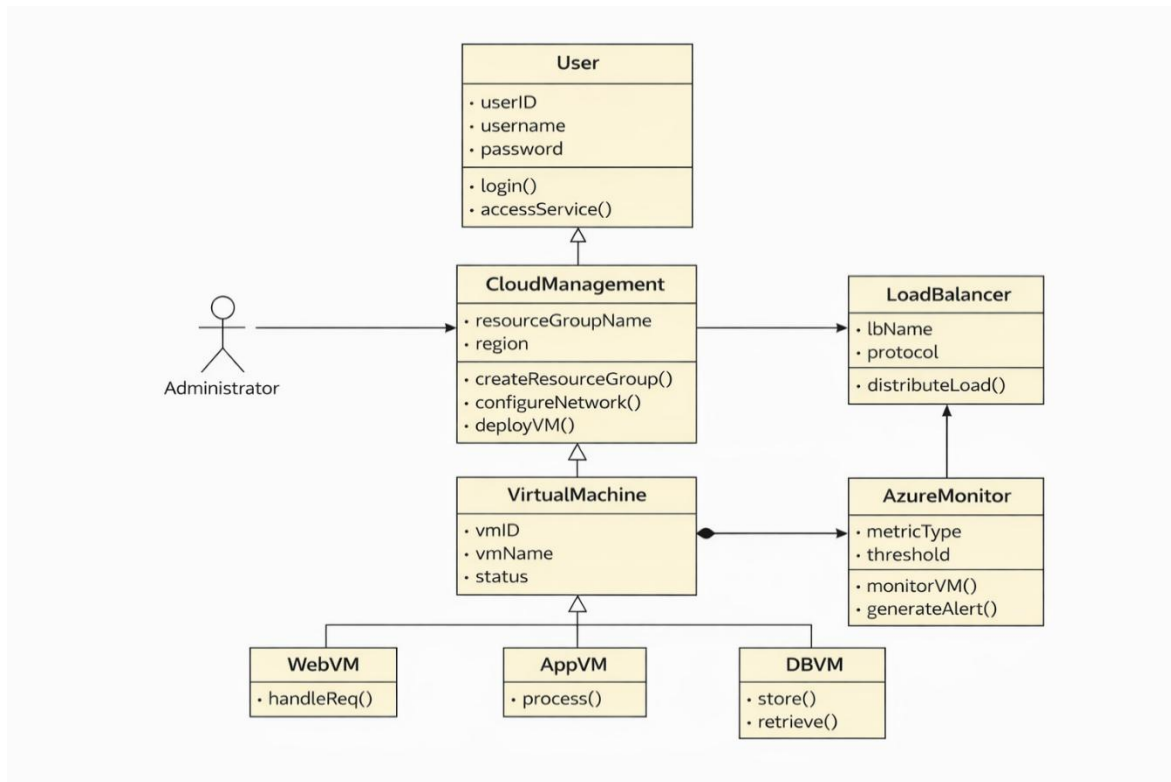


Fig 4.2: Class diagram

EXPLANATION

The User accesses application services through the cloud system.

The Administrator manages cloud resources using the Cloud Management class.

Virtual Machine represents servers in the system and is divided into WebVM, AppVM, and DBVM for different tiers.

The Load Balancer distributes incoming requests to maintain system performance.

Azure Monitor monitors system performance and generates alerts when resource usage exceeds the threshold

4.1.3 OBJECT DIAGRAM

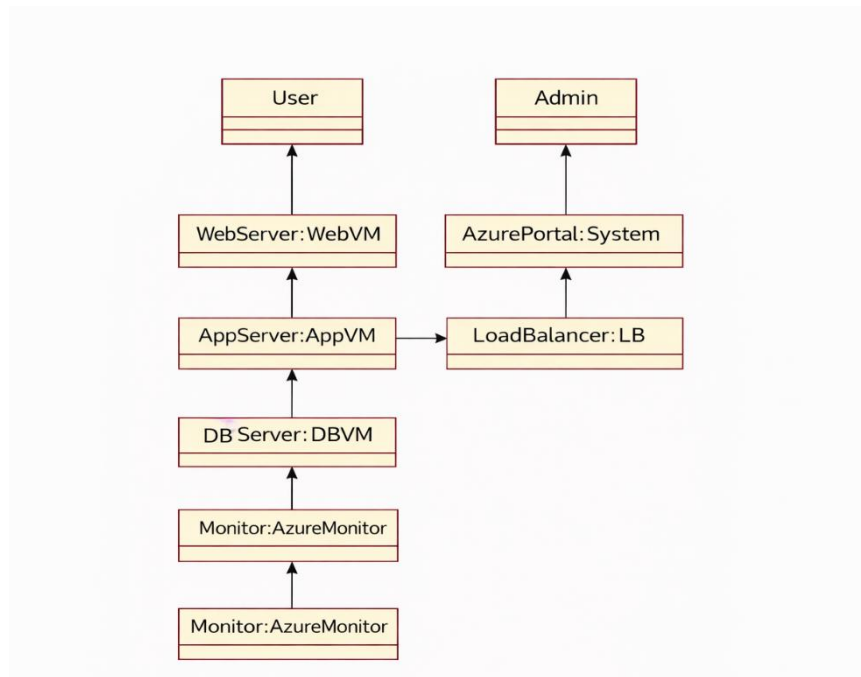


Fig 4.3: Object diagram

EXPLANATION:

The object diagram represents the instances of classes involved in the multi-tier cloud system. The Admin manages the cloud infrastructure through the Azure portal, while the User accesses the deployed application services. The WebServer, AppServer, and DBServer objects represent the virtual machines used for the web, application, and database tiers. The Load Balancer distributes incoming requests to maintain system performance. The Azure Monitor object tracks system performance and generates alerts when resource usage exceeds the defined threshold.

4.1.4 STATE CHART DIAGRAM

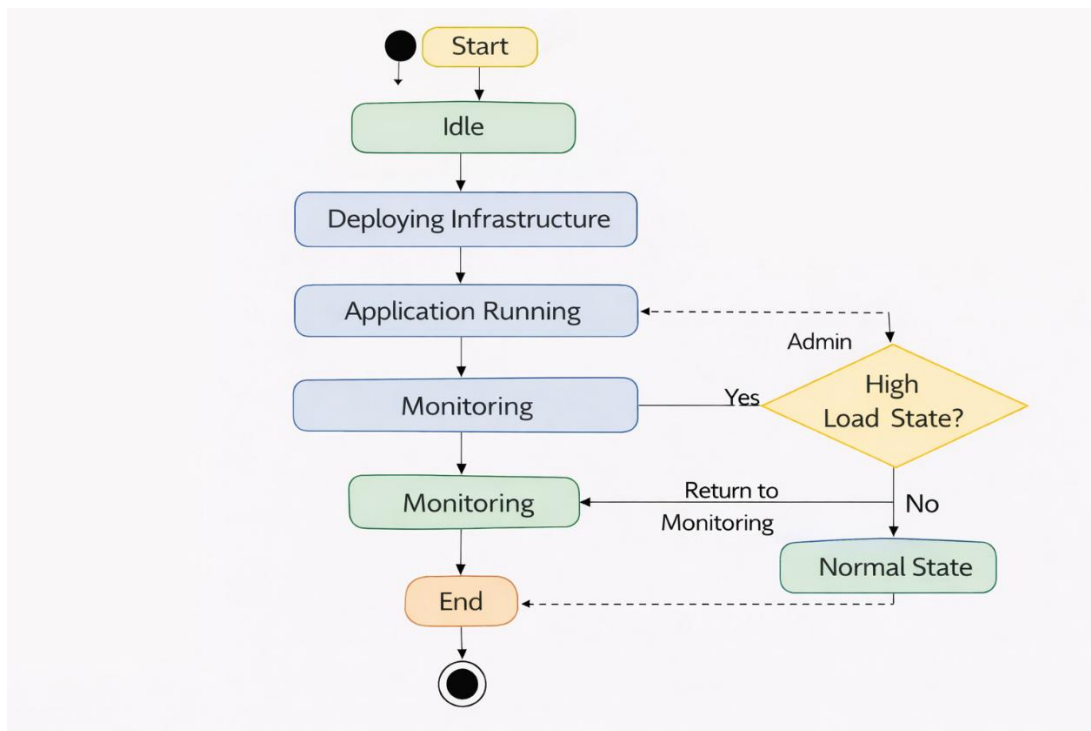


Fig 4.4: State Chart diagram

EXPLANATION:

The process starts when the administrator logs into the Azure portal and creates a Resource Group. Then the Virtual Network and subnets are configured to enable communication between servers. Virtual Machines (WebVM, AppVM, DBVM) are deployed and a Load Balancer is set up to distribute user requests.

The application starts running and users access the services through the system.

Azure Monitor checks system performance, and if resource usage exceeds the threshold, an alert is generated for the administrator.

4.1.5 SEQUENCE DIAGRAM

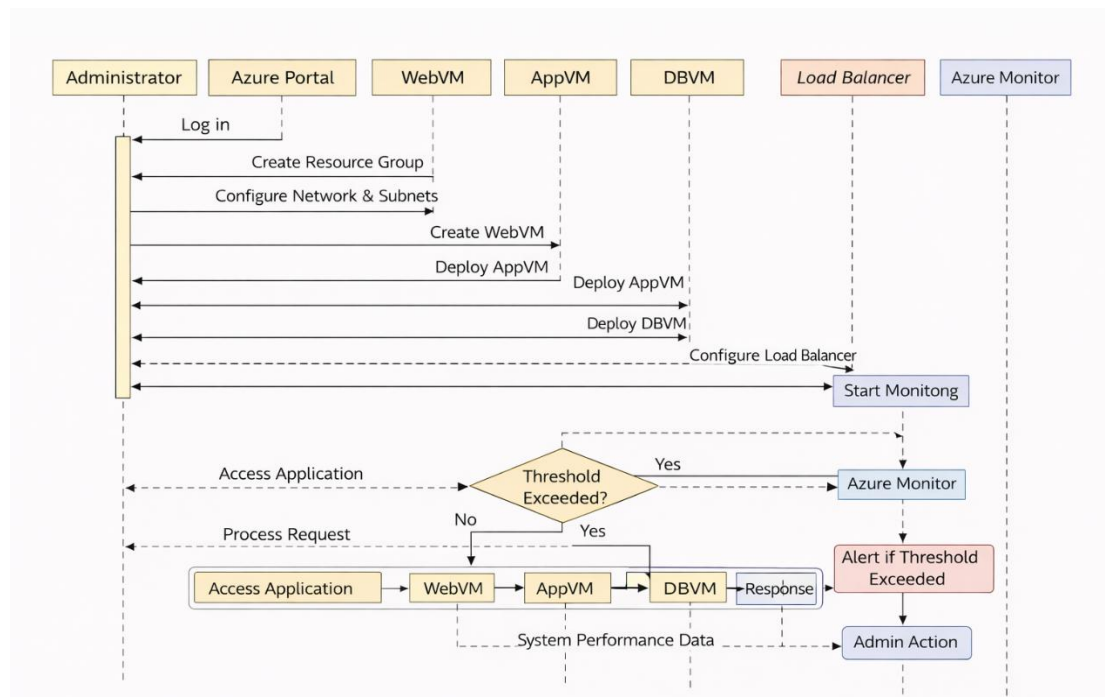


Fig 4.5: Sequence diagram

EXPLANATION:

The sequence diagram shows how different components interact during the deployment and operation of the multi-tier application. The Administrator logs into the Azure portal, creates the resource group, and configures the virtual network and subnets. Then the system deploys WebVM, AppVM, and DBVM and configures the load balancer. Users access the application through the WebVM, which communicates with AppVM and DBVM to process requests. Meanwhile, Azure Monitor tracks system performance and generates alerts if the resource usage exceeds the defined threshold, allowing the administrator to take necessary action.

4.1.6 COLLABORATION DIAGRAM

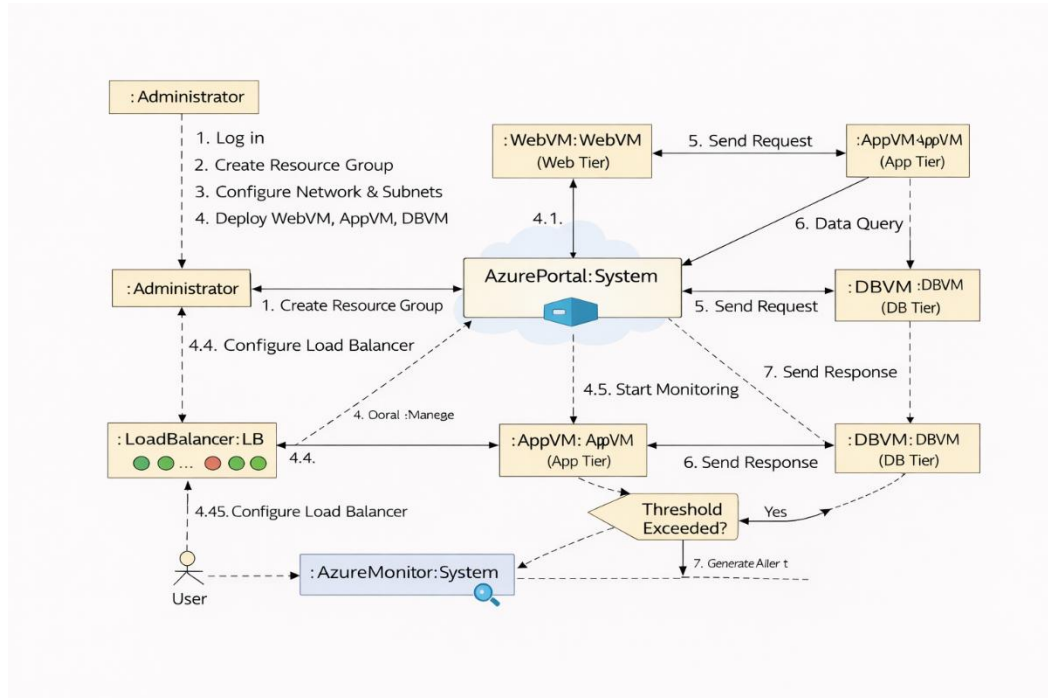


Fig 4.6: Collaboration diagram

EXPLANATION:

The collaboration diagram shows how different components of the multi-tier cloud system interact with each other. The Administrator logs into the Azure portal, creates the resource group, configures the network and subnets, and deploys WebVM, AppVM, and DBVM. The Load Balancer is configured to distribute incoming user requests to the web server. The WebVM communicates with AppVM, which retrieves data from the DBVM and sends the response back to the user. Meanwhile, Azure Monitor continuously observes system performance, and if the resource usage exceeds the threshold, an alert is generated so that the administrator can take corrective action.

4.1.7 ACTIVITY DIAGRAM

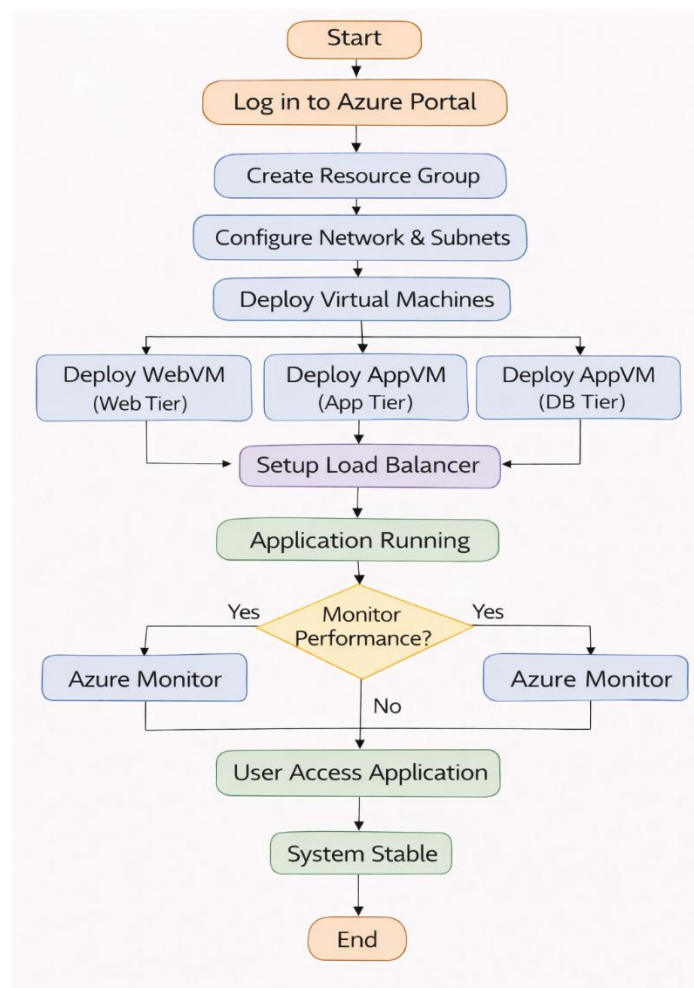


Fig 4.7: Activity diagram

EXPLANATION:

The activity diagram represents the workflow for deploying and managing a multi-tier application in Microsoft Azure. The process starts when the administrator logs into the Azure portal and creates a resource group. Then the virtual network and subnets are configured, followed by deploying the virtual machines for the web, application, and database tiers. After that, a load balancer is set up and the application starts running. Azure Monitor checks system performance, and if the system works normally, users can access the application and the system remains stable.

1.1.8 COMPONENT DIAGRAM

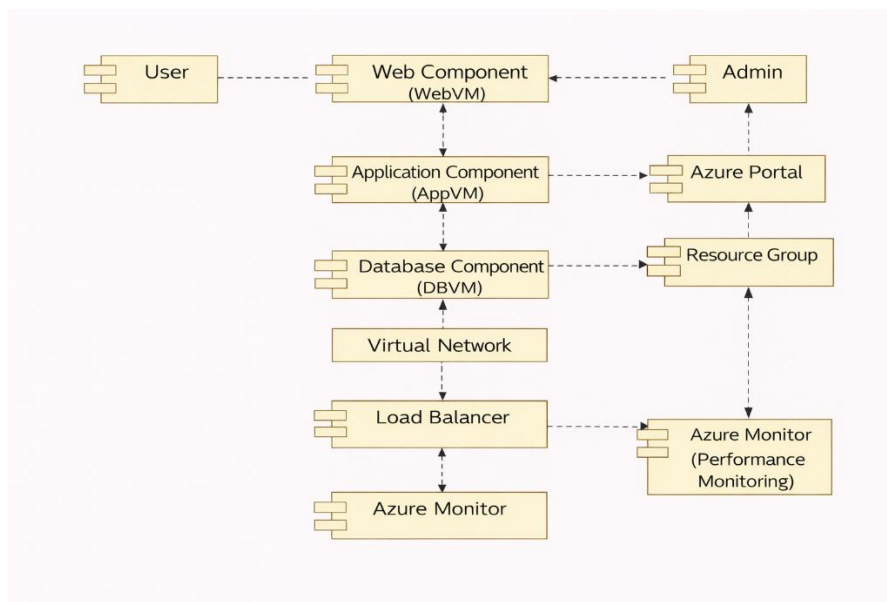


Fig 4.8: Component diagram

EXPLANATION:

The component diagram shows the main components of the multi-tier cloud application system and how they interact. The User accesses the application through the WebVM, which sends requests to the AppVM for processing. The AppVM communicates with the DBVM to store or retrieve data. The Administrator manages the infrastructure through the Azure Portal by configuring resources and deploying servers. The Load Balancer distributes incoming traffic, while Azure Monitor tracks system performance and generates alerts if the system exceeds defined thresholds.

4.1.9 DATA FLOW DIAGRAM

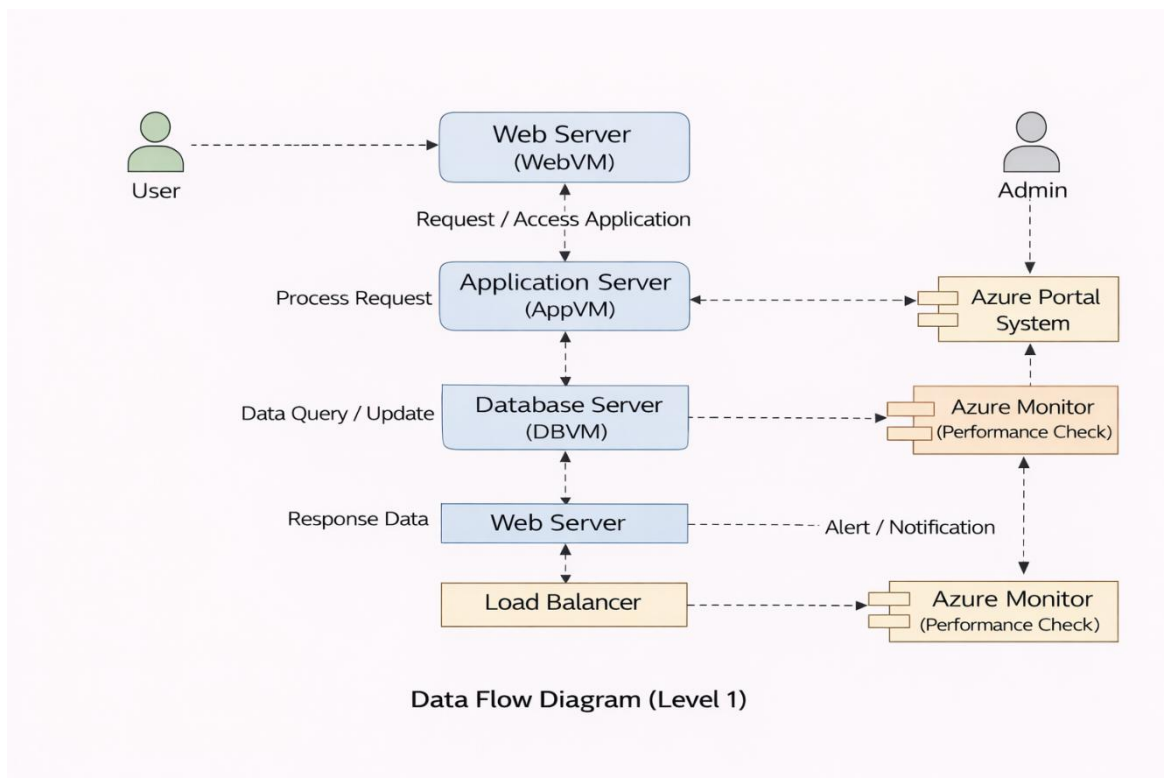


Fig 4.9: Data flow diagram

EXPLANATION:

The Data Flow Diagram shows how data moves within the multi-tier cloud application system. The User sends requests to the Web Server (WebVM) to access the application. The request is processed by the Application Server (AppVM), which interacts with the Database Server (DBVM) to retrieve or store data. The response is then sent back to the user through the web server. The Administrator manages the system through the Azure Portal, while Azure Monitor collects performance metrics and generates alerts if system thresholds are exceeded.

4.1.10 DEPLOYMENT DIAGRAM

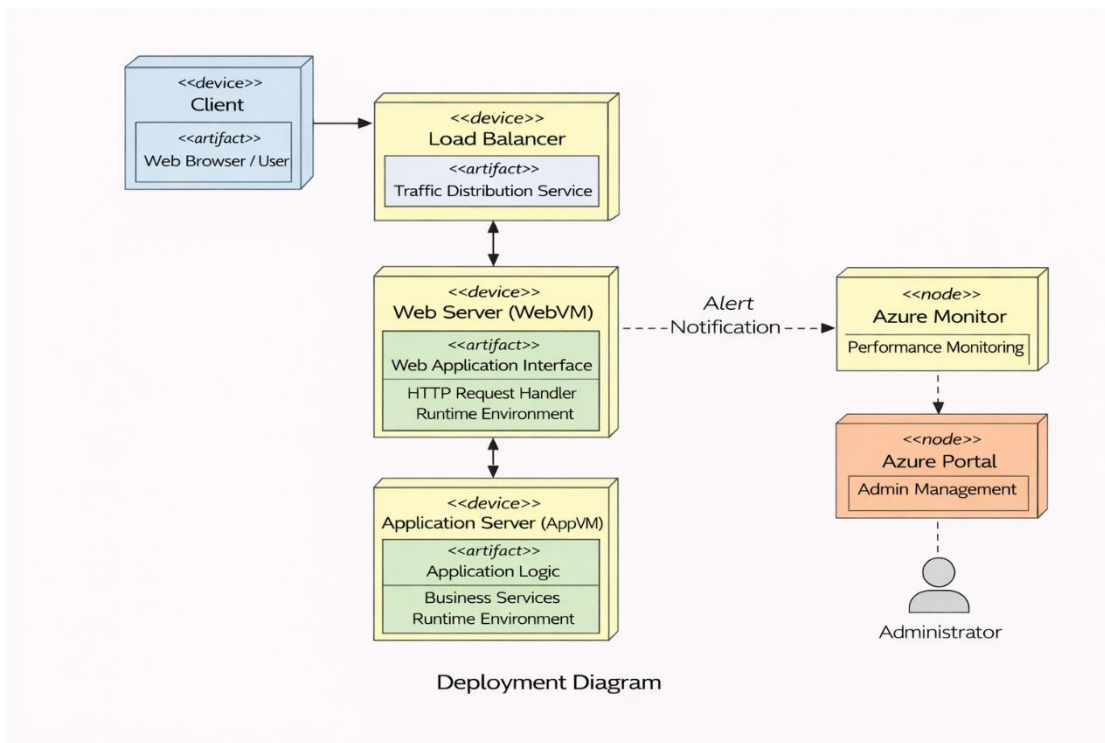


Fig 4.10: Deployment diagram

EXPLANATION:

The deployment diagram shows how the multi-tier application is physically deployed in the Azure cloud environment. The client (user) accesses the system through a web browser, and requests are sent to the Load Balancer, which distributes traffic to the Web Server (WebVM). The web server communicates with the Application Server (AppVM) to process requests and retrieve data from the Database Server (DBVM). The Administrator manages the infrastructure through the Azure Portal, while Azure Monitor continuously tracks system performance and generates alerts when thresholds are exceeded.

1.2 SYSTEM ARCHITECTURE

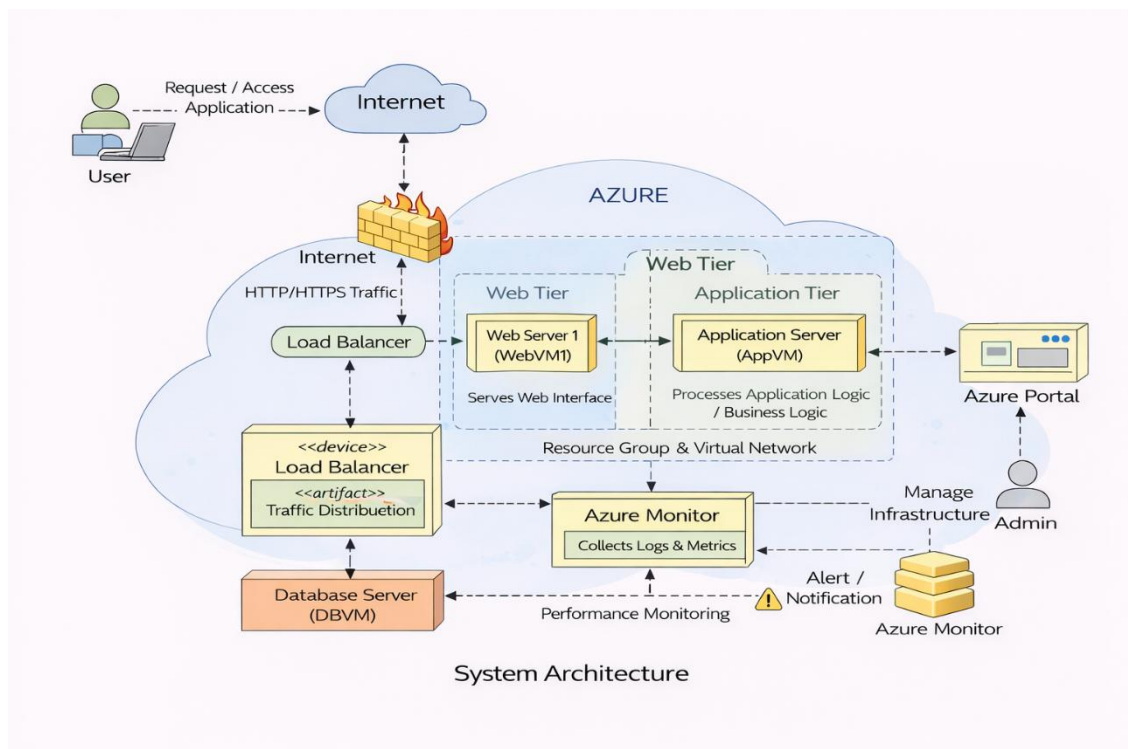


Fig 4.11: System Architecture

The system architecture shows how the multi-tier application is organized in the Azure cloud environment. The user sends requests through the internet, which are directed to the Load Balancer. The load balancer distributes the requests to the Web Server (WebVM) that provides the web interface. The Application Server (AppVM) processes the business logic and communicates with the Database Server (DBVM) to store or retrieve data. The Administrator manages the system using the Azure Portal, while Azure Monitor tracks performance and generates alerts when system thresholds are exceeded.

CHAPTER 5

DEVELOPMENT TOOLS

5.1 GENERAL

This chapter describes the technologies, cloud services, and tools used for implementing the proposed system. The project is developed using Microsoft Azure cloud platform to deploy and manage a multi-tier application architecture with automated monitoring and security policies.

Modern applications require scalable infrastructure, continuous monitoring, and strong security mechanisms. Microsoft Azure provides a wide range of cloud services that enable developers to deploy applications efficiently while ensuring reliability, high availability, and secure communication between system components.

The major objectives of using Azure cloud services in this project are:

- Deploy multi-tier applications in a cloud environment
- Enable automated monitoring of system resources
- Implement security policies for protecting application data
- Ensure scalability and high availability of application services
- Reduce infrastructure management overhead

Using Azure cloud infrastructure eliminates the need for physical servers and enables dynamic resource allocation based on workload requirements.

5.2 CLOUD PLATFORM AND SERVICES

5.2.1 Microsoft Azure

Microsoft Azure is a leading cloud computing platform that provides Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). It enables organizations to build, deploy, and manage applications through Microsoft-managed data centers.

Azure supports the deployment of scalable and secure multi-tier applications by offering services for computing, networking, storage, monitoring, and security management.

In this project, Azure is used to:

- Deploy virtual machines for application components
- Configure network infrastructure for multi-tier architecture
- Implement load balancing for traffic distribution
- Monitor system performance and generate alerts
- Enforce security policies for controlled access

5.2.2 Azure Services Used

1. Azure Virtual Machines

Azure Virtual Machines (VMs) provide scalable computing resources that allow users to deploy applications on virtual servers.

In this project, virtual machines are used to implement the three-tier architecture:

- **Web Server (WebVM)** – Handles incoming user requests
- **Application Server (AppVM)** – Executes business logic
- **Database Server (DBVM)** – Stores application data

These virtual machines run the Ubuntu operating system and communicate through a virtual network.

2. Azure Virtual Network (VNet)

Azure Virtual Network provides a secure network environment where different application components can communicate with each other.

In this project, the virtual network contains three subnets:

- **WebSubnet** – Hosts the web server
- **AppSubnet** – Hosts the application server
- **DBSubnet** – Hosts the database server

This separation ensures better security and controlled communication between different application tiers.

3. Azure Load Balancer

Azure Load Balancer distributes incoming traffic across multiple servers to improve application availability and reliability.

In this project, the load balancer directs user requests to the web server instances. This prevents server overload and ensures efficient traffic management.

Benefits include:

- Improved performance
- High availability
- Automatic traffic distribution

4. Azure Monitor

Azure Monitor is used to collect and analyze performance metrics from cloud resources.

It continuously tracks:

- CPU usage
- Memory utilization
- Network traffic
- System logs

In this project, Azure Monitor generates alerts when system performance exceeds predefined thresholds. For example, an alert is triggered when CPU utilization exceeds 80%, enabling administrators to take corrective action.

5. Azure Security Features

Azure provides multiple security mechanisms to protect application resources.

Key security features used in this project include:

Authentication and Access Control

- Secure login for authorized users

Network Security

- Control of inbound and outbound traffic

Role-Based Access Control (RBAC)

- Restricts user permissions based on roles

These security mechanisms help protect the system from unauthorized access and ensure safe communication between different application tiers.

5.2.3 Multi-Tier Architecture in Azure

The project follows a **three-tier architecture** deployed on Microsoft Azure.

Presentation Layer (Web Tier)

This layer interacts with users through the web interface. It receives user requests and forwards them to the application tier.

Application Layer (Business Logic Tier)

This layer processes user requests, executes application logic, and communicates with the database layer.

Data Layer (Database Tier)

This layer stores application data, logs, and configuration information. It ensures data integrity and secure storage.

This layered architecture improves system scalability, maintainability, and security.

5.3 CONCLUSION

Microsoft Azure provides a reliable and scalable platform for deploying and managing multi-tier applications. By using Azure Virtual Machines, Virtual Networks, Load Balancers, and Azure Monitor, the system achieves automated deployment, continuous monitoring, and enhanced security.

The cloud-based architecture enables efficient resource utilization, improved application performance, and real-time monitoring of system health. As a result, the proposed system ensures high availability, strong security enforcement, and efficient management of multi-tier applications in a cloud environment.

CHAPTER 6

IMPLEMENTATION

6.1 GENERAL

The implementation phase focuses on deploying the proposed multi-tier application architecture using Microsoft Azure cloud services. The system is designed to demonstrate how application components can be organized into separate layers and managed efficiently in a cloud environment.

The architecture follows a three-tier structure consisting of the web tier, application tier, and database tier. Each tier is deployed on separate virtual machines to improve scalability and simplify management. These servers communicate through an Azure Virtual Network that ensures secure connectivity between the system components.

The implementation also includes monitoring and security mechanisms. Performance monitoring is carried out using Azure Monitor, which continuously tracks system metrics and generates alerts when abnormal activity is detected. This helps administrators maintain system reliability and quickly respond to potential issues.

6.2 IMPLEMENTATION

Technologies and Tools Used:

The implementation of the proposed system uses several tools and cloud services provided by Microsoft Azure. These technologies help in deploying, managing, and monitoring the multi-tier application infrastructure.

Microsoft Azure

Microsoft Azure is the cloud platform used to deploy and manage all project resources. It provides infrastructure services that allow users to create virtual machines, configure networks, and monitor system performance.

Azure Virtual Machines

Azure Virtual Machines provide virtual servers that host the application components. In this project, three virtual machines are created to represent the web server, application server, and database server.

Azure Virtual Network

The Azure Virtual Network provides a secure network environment that connects all the servers used in the project. It allows communication between the web, application, and database tiers.

Subnets

Subnets are created within the virtual network to separate different layers of the system. The project uses three subnets:

- Web Subnet
- Application Subnet
- Database Subnet

This structure improves system organization and security.

Azure Load Balancer

Azure Load Balancer distributes incoming traffic across available web servers. It helps improve system performance and ensures that no single server becomes overloaded.

Azure Monitor

Azure Monitor is used to observe system performance and resource utilization. It collects metrics such as CPU usage and network activity. Alerts are generated automatically when predefined thresholds are exceeded.

Implementation Workflow:

1. Login to Azure Portal

Access the Microsoft Azure portal using a web browser and sign in with a valid Azure account.

2. Create Resource Group

Create a resource group named *MultiTierRG* to store and manage all the project resources.

3. Create Virtual Network

Configure a Virtual Network (VNet) to enable communication between different components of the system.

4. Configure Subnets

Create three subnets within the virtual network: WebSubnet, AppSubnet, and DBSubnet.

5. Deploy Virtual Machines

Deploy WebVM, AppVM, and DBVM in their respective subnets to represent the web, application, and database tiers.

6. Configure Load Balancer

Set up an Azure Load Balancer to distribute incoming traffic across the web servers.

7. Enable Monitoring

Use Azure Monitor to track system performance and configure alerts when CPU usage exceeds the defined threshold.

CHAPTER 7

SNAPSHOTS

7.1 GENERAL

The project is designed as a **multi-tier application management framework** integrating automated deployment, real-time monitoring, and security policy enforcement. The system architecture consists of three main tiers:

- **Web Tier:** Handles user requests, authentication, and provides the interface for interacting with the application.
- **Application Tier:** Executes business logic, manages workflows, and coordinates communication between the web and database tiers.
- **Database Tier:** Stores application data securely, enforces access control, and ensures compliance with security policies.

The framework supports **dynamic deployment**, where application components are allocated to appropriate servers based on resource availability, inter-tier dependencies, and system load.

7.2 VARIOUS SNAPSHOTS

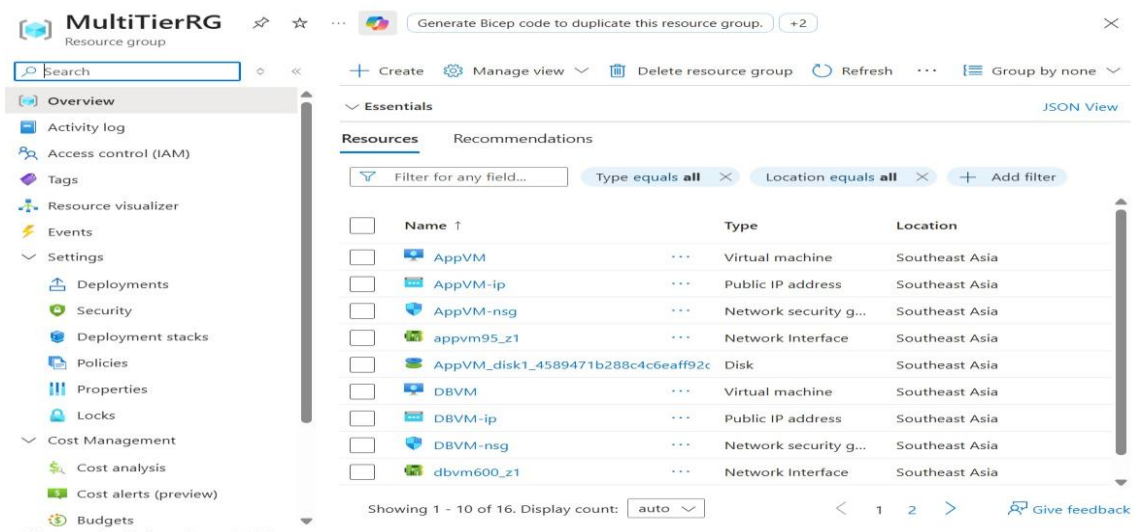


Fig 7.1: Create Resource Group

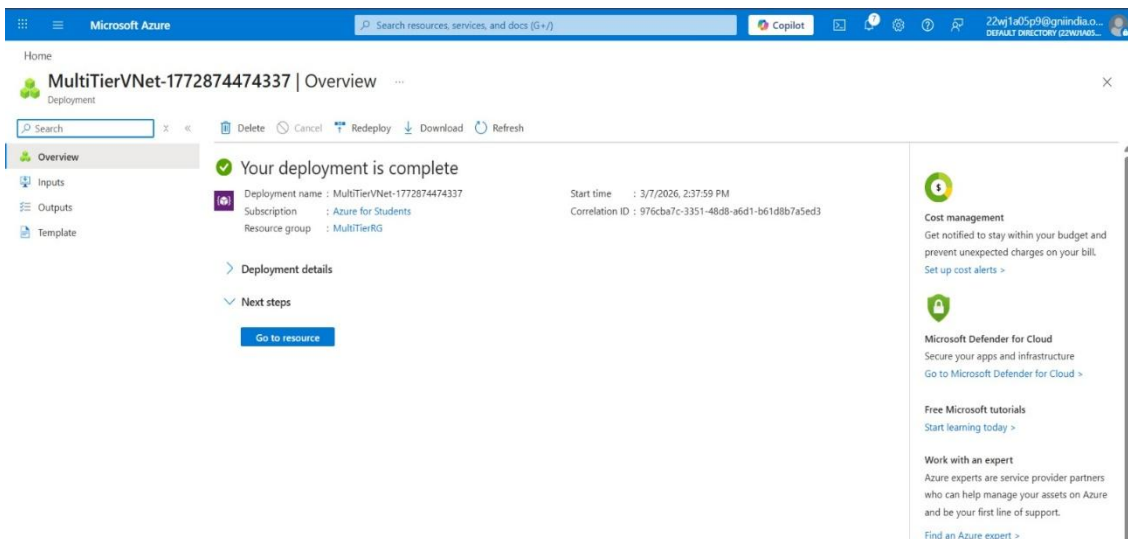


Fig 7.2: Create Virtual Network

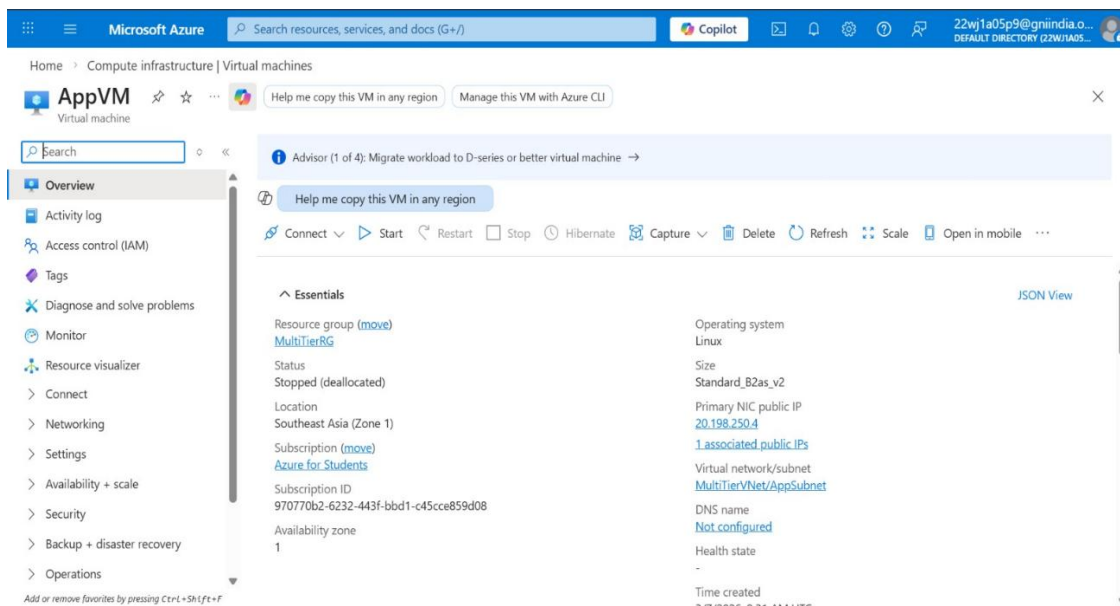


Fig 7.3: Create AppVM Virtual Machine

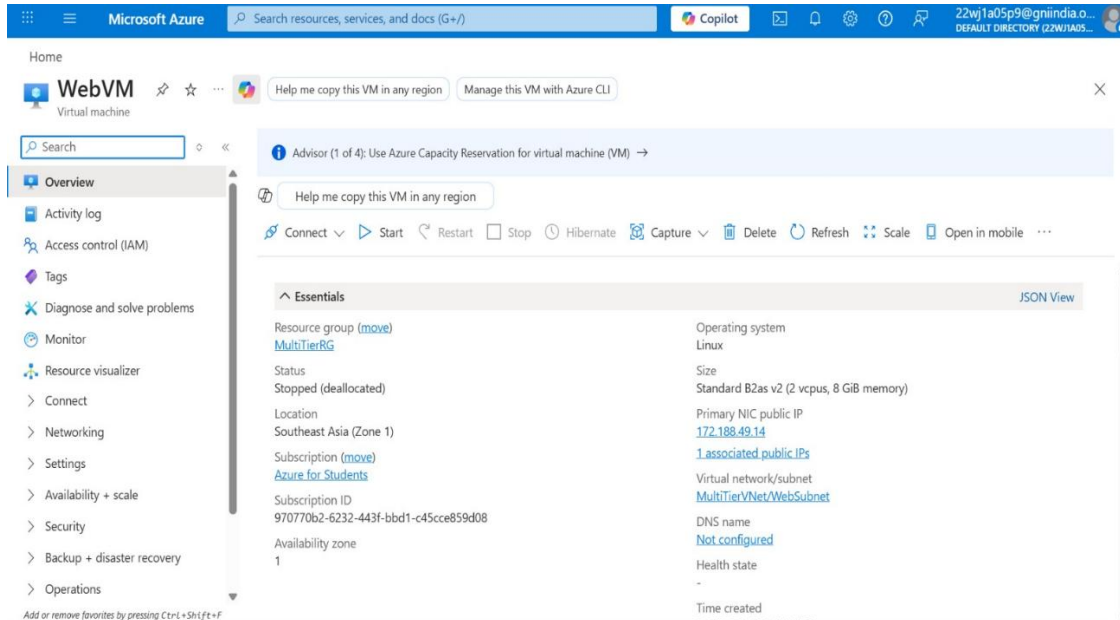


Fig 7.4: Create WebVM Virtual Machine

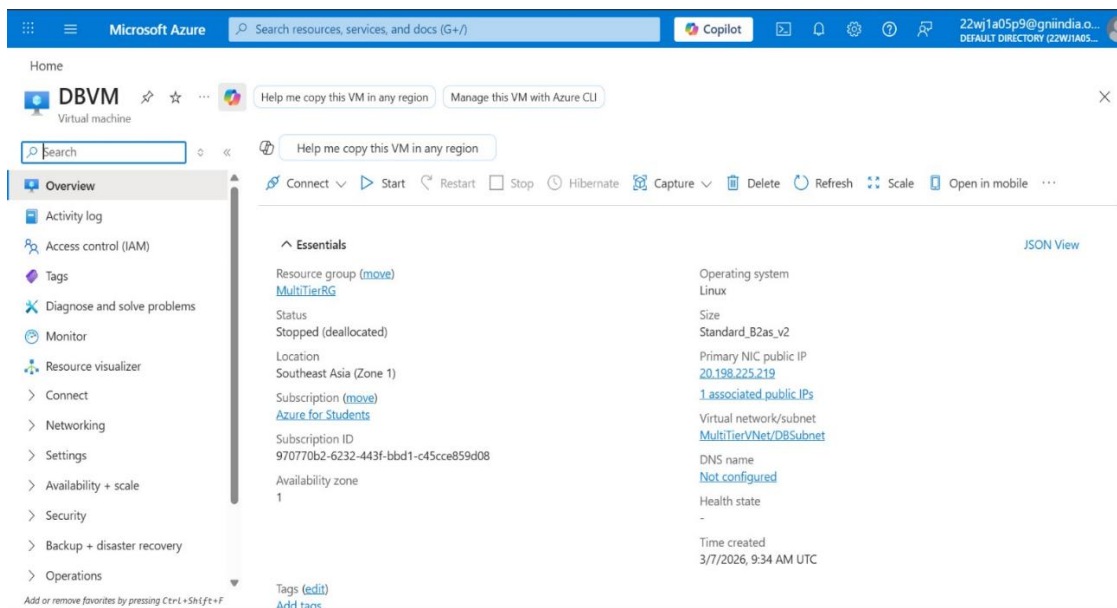


Fig 7.5: Create DBVM Virtual Machine

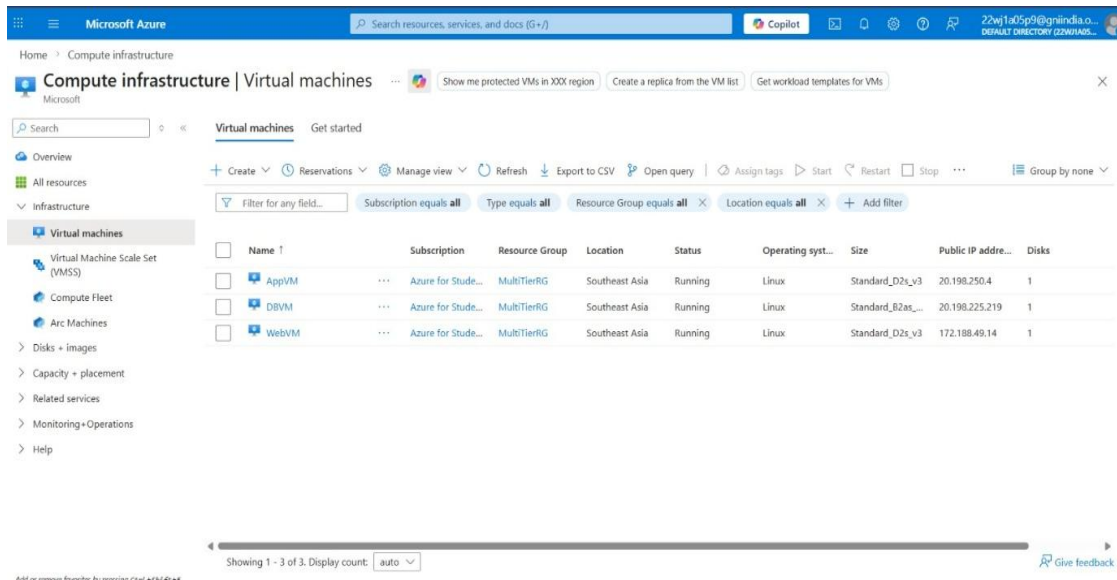
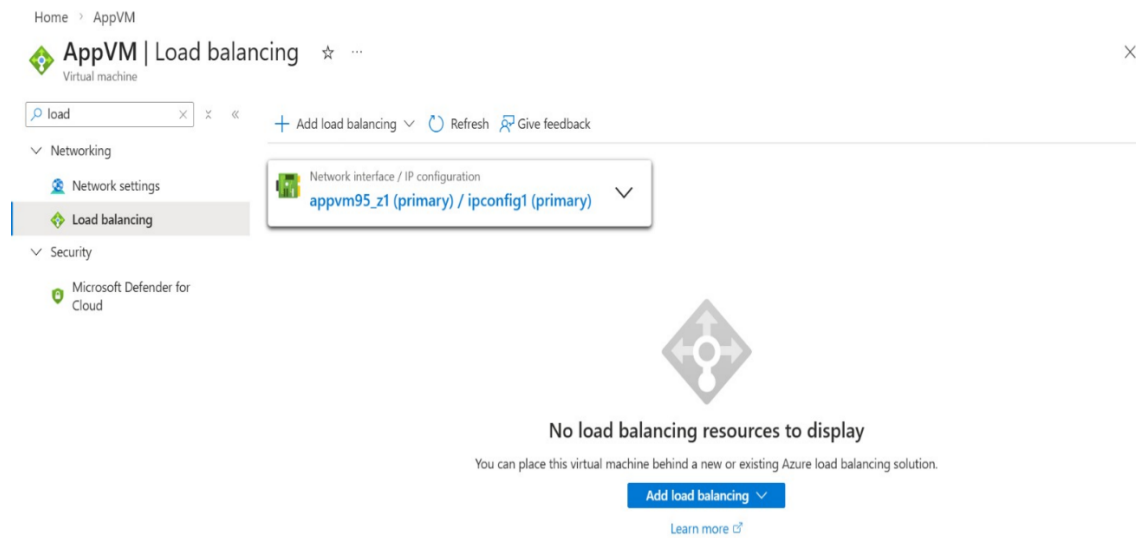


Fig 7.6: Deploy Virtual Machines



Fig

7.7: Configure Load Balancer

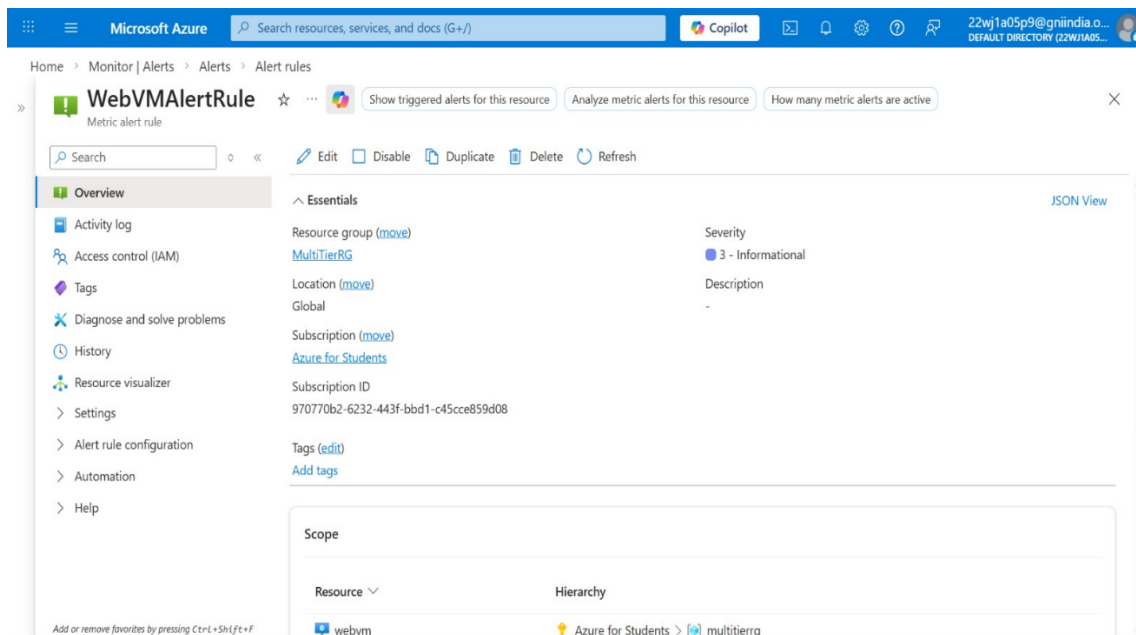


Fig 7.8: Performance Monitoring

CHAPTER 8

SOFTWARE TESTING

8.1 GENERAL

The purpose of software testing is to identify errors, validate functionality, and ensure that the system operates as expected under all conditions. In the context of **multi-tier applications**, testing ensures that all tiers—web, application, and database—work together seamlessly, meet performance requirements, comply with security policies, and maintain reliability under varying workloads. Testing provides confidence that the deployment, monitoring, and security enforcement components of the system are functioning correctly and do not fail under normal or exceptional scenarios.

8.2 DEVELOPING METHODOLOGIES

The testing methodology used in this project follows a structured approach to verify the functionality and performance of the system. The testing process begins by validating individual components such as virtual machines and network configuration. After verifying individual components, the interaction between different modules is tested.

The methodology includes:

1. Verifying that each cloud component is deployed successfully.
2. Testing communication between the web, application, and database tiers.
3. Checking load balancing functionality.
4. Monitoring system performance using Azure Monitor.
5. Ensuring that alerts are generated when system thresholds are exceeded.

This approach helps ensure that the system operates reliably and efficiently.

8.3 TYPES OF TESTING

8.3.1 Unit Testing

Unit testing focuses on testing individual components of the system separately. In this project, unit testing is used to verify that each Azure resource such as virtual machines, virtual network, and subnets is created and configured correctly.

For example, the WebVM, AppVM, and DBVM are tested individually to ensure that they are deployed successfully and operating properly.

8.3.2 Functional Testing

Functional testing verifies whether the system performs its intended operations. In this project, functional testing ensures that the multi-tier architecture works correctly and that requests from users are processed properly.

This includes verifying that the web server receives user requests, the application server processes the requests, and the database server stores and retrieves data correctly.

8.3.3 System Testing

System testing evaluates the complete system as a whole. In this stage, all the components including virtual machines, load balancer, and monitoring services are tested together to ensure that they work as an integrated system.

The goal is to confirm that the deployed cloud infrastructure supports the multi-tier application effectively.

8.3.4 Performance Testing

Performance testing checks how well the system performs under different workloads. This testing evaluates system response time, resource usage, and the ability to handle multiple requests.

Azure Monitor is used to observe system metrics such as CPU utilization and network traffic during performance testing.

8.3.5 Integration Testing

Integration testing verifies that different modules of the system interact correctly with each other. In this project, integration testing ensures that the web tier communicates with the application tier and that the application tier interacts properly with the database tier.

Successful integration ensures smooth communication between system components.

8.3.6 Acceptance Testing

Acceptance testing is performed to confirm that the system meets the project requirements and operates according to the expected functionality. The deployed system is evaluated to ensure that the multi-tier architecture, monitoring system, and security mechanisms work as intended. Once the system successfully passes acceptance testing, it is considered ready for deployment.

8.3.7 Build the test plan

Any project can be divided into units that can be further performed for detailed processing. Then a testing strategy for each of this unit is carried out. Unit testing helps to identify the possible bugs in the individual component, so the component that has bugs can be identified and can be rectified from errors.

CHAPTER 9

APPLICATION

9.1 FUTURE ENHANCEMENT

APPLICATION FUTURE ENHANCEMENT

Future work on the **AutoDeploy** framework will focus on enhancing **scalability, automation, and security compliance** for multi-tier applications in dynamic cloud and edge environments.

Potential enhancements include:

1. **Advanced AI-Driven Resource Management:** Incorporating reinforcement learning and predictive analytics to optimize deployment decisions in real time, considering changing workloads, inter-tier dependencies, and server capacities.
2. **Enhanced Monitoring and Anomaly Detection:** Developing more sophisticated monitoring algorithms capable of detecting subtle performance degradations, security threats, or policy violations automatically, reducing human intervention.
3. **Adaptive Security Policy Enforcement:** Implementing context-aware and dynamic security policies that adjust based on application criticality, user behavior, and network conditions, ensuring compliance without affecting performance.
4. **Hybrid Deployment Strategies:** Exploring hybrid deployment across cloud and edge servers to balance latency, cost, and resource utilization, particularly for latency-sensitive applications such as AR/VR, AI services, and microservices-based architectures.
5. **Predictive Scaling and Fault Tolerance:** Integrating predictive scaling mechanisms to anticipate spikes in user demand and automatically reallocate resources, along with improved fault-tolerance strategies for high availability in multi-tier deployments.

CHAPTER 10

CONCLUSION & REFERENCE

10.1 CONCLUSION

This project demonstrates the deployment and management of a multi-tier application architecture using Microsoft Azure cloud services. The system is organized into three layers: the web tier, application tier, and database tier, each deployed on separate virtual machines to improve scalability and maintainability.

Azure services such as Virtual Network, Subnets, Load Balancer, and Azure Monitor are used to create a secure and efficient cloud infrastructure. The load balancer distributes incoming traffic across servers to maintain system performance, while Azure Monitor tracks system metrics and generates alerts when performance thresholds are exceeded.

Overall, the project highlights how cloud platforms can simplify application deployment, improve system reliability, and support automated monitoring and security management. The implementation demonstrates a practical approach to managing scalable and efficient cloud-based applications.

10.2 REFERENCE

- AWS, “AWS Wavelength for media & entertainment,” 2021. [Online]. Available: <https://d1.awsstatic.com/Wavelength2020/AWS-Wavelength-for-Media-Entertainment-SolutionBrief-Feb2021-Final.pdf>
- G. Zhu et al., “Pushing AI to wireless network edge: An overview on integrated sensing, communication, and computation towards 6G,” *Sci. China Inf. Sci.*, vol. 66, no. 3, 2023, Art. no. 130301.
- G. Cheng, D. Guo, L. Luo, J. Xia, and S. Gu., “AutoDeploy: A lightweight strategy for efficient multi-tier application deployment with automated monitoring and security policies,” *IEEE Trans. Serv. Comput.*, 2023.
- R. Li, Z. Zhou, X. Zhang, and X. Chen, “Joint application placement and request routing optimization for dynamic edge computing service management,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4581–4596, Dec. 2022.
- W. Liu, Y. Guo, and X. Shen, “Delay aware microservice coordination in mobile edge computing: A reinforcement learning approach,” *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.
- Q. He, Z. Dong, F. Chen, S. Deng, W. Liang, and Y. Yang, “Pyramid: Enabling hierarchical neural networks with edge computing,” in *Proc. ACM WebConf.*, 2022, pp. 1860–1870.
- H. Jin, R. Luo, Q. He, S. Wu, Z. Zeng, and X. Xia, “Cost-effective application placement with resource and security constraints in edge computing systems,” *IEEE Trans. Serv. Comput.*, vol. 16, no. 2, pp. 1039–1050, Mar./Apr. 2023.
- X. Xia, F. Chen, Q. He, J. Grundy, M. Abdelrazek, and H. Jin, “Online collaborative deployment and monitoring in edge computing environments,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 281–294, Feb. 2021.
- X. Ge, S. Tu, G. Mao, C.-X. Wang, and T. Han, “5G ultra-dense cellular networks for application-aware edge deployment,” *IEEE Wireless Commun.*, vol. 23, no. 1, pp. 72–79, Feb. 2016.
- S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, “AI-driven adaptive resource management for multi-tier applications in edge computing,” *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.

- J. Yang, A. Sabnis, D. S. Berger, K. Rashmi, and R. K. Sitaraman, “C2DN: Efficient content delivery and deployment in multi-tier cloud-edge environments,” in *Proc. USENIX Symp. Netw. Syst. Des. Implementation*, 2022, pp. 1159–1177.
- Z. Liu et al., “DistCache: Load balancing for large-scale application deployment in edge computing,” in *Proc. USENIX Conf. File Storage Technol.*, 2019, pp. 143–157.
- X. Zou, W. Xia, P. Shilane, H. Zhang, and X. Wang, “High-performance deployment framework for edge applications with AI-based monitoring,” in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 19–36.
- Y. Shin, D. Koo, and J. Hur, “A survey of secure and policy-compliant application deployment frameworks,” *ACM Comput. Surv.*, vol. 49, no. 4, pp. 1–38, 2017.
- X. Zou, J. Yuan, P. Shilane, W. Xia, H. Zhang, and X. Wang, “Balancing deployment efficiency and monitoring overhead in multi-tier applications,” in *Proc. USENIX Conf. File Storage Technol.*, 2021, pp. 171–185.
- Z. Cao, S. Liu, F. Wu, G. Wang, B. Li, and D. H. Du, “Optimized deployment strategies for latency-sensitive applications in edge networks,” in *Proc. USENIX Conf. File Storage Technol.*, 2019, pp. 129–142.
- W. Xia et al., “A comprehensive study of multi-tier application management frameworks in edge computing,” *Proc. IEEE*, vol. 104, no. 9, pp. 1681–1710, Sep. 2016.
- W. Y. B. Lim et al., “Decentralized AI-based application management for hierarchical edge computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 536–550, Mar. 2022.
- R. Kübler, “Popularity-aware application deployment and caching in edge networks,” 2023.
- F. Ni and S. Jiang, “Rapid adaptation for multi-tier application scaling in edge environments,” in *Proc. ACM Symp. Cloud Comput.*, 2019, pp. 220–232.
- R. Kisous, A. Kolikant, A. Duggal, S. Sheinvald, and G. Yadgar, “Migration strategies for multi-tier applications in dynamic edge networks,” *ACM Trans. Storage*, vol. 18, no. 4, pp. 1–29, 2022.
- R. Luo, H. Jin, Q. He, S. Wu, and X. Xia, “Load-balanced deployment of multi-tier applications in edge computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1420–1431, May 2023.
- J. Gao, S. Zhang, L. Zhao, and X. S. Shen, “Dynamic probabilistic deployment for time-varying workloads,” *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1672–1684, Apr. 2021.

- Y. Li, C. Tian, F. Guo, C. Li, and Y. Xu, “Elastic monitoring strategies for large-scale multi-tier applications,” in *Proc. USENIX Annu. Tech. Conf.*, 2019, pp. 739–752.
- L. Fan, P. Cao, J. Almeida, and A. Z. Broder, “SummaryCache: Scalable caching for multi-tier cloud applications,” *IEEE/ACM Trans. Netw.*, vol. 8, no. 3, pp. 281–293, Jun. 2000.
- A. Pagh, R. Pagh, and S. S. Rao, “Optimal filtering techniques for large-scale application monitoring,” in *Proc. Annu. ACM-SIAM Symp. Discrete Algorithms*, 2005, pp. 823–829.
- N. Dayan, I. Bercea, P. Reviriego, and R. Pagh, “InfiniFilter: Scalable monitoring filters for multi-tier applications,” in *Proc. ACM Int. Conf. Manage. Data*, 2023, pp. 1–27
- P. Pandey, A. Conway, J. Durie, M. A. Bender, M. Farach-Colton, and R. Johnson, “Vector quotient filters for efficient deployment monitoring,” in *Proc. ACM Int. Conf. Manage. Data*, 2021, pp. 1386–1399.
- F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, “Survey on edge computing tools for multi-tier application deployment and management,” *Proc. IEEE*, vol. 107, no. 8, pp. 1537–1562, Aug. 2019.
- S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, “Delay-aware coordination for microservices in mobile edge computing,” *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.
- [26] P. Yang, N. Zhang, S. Zhang, L. Yu, J. Zhang, and X. S. Shen, “Content popularity prediction towards location-aware mobile edge caching,” *IEEE Trans. Multimedia*, vol. 21, no. 4, pp. 915–929, Apr. 2019.
- [27] J. Gao, S. Zhang, L. Zhao, and X. S. Shen, “The design of dynamic probabilistic caching with time-varying content popularity,” *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1672–1684, Apr. 2021.